

Advanced Cloud Deployment:

High-Availability LAMP Stack Architecture on OpenStack

Environment: MicroStack (Ubuntu 24.04 nodes)

1. Executive Summary & Architecture

This report documents the end-to-end engineering, troubleshooting, and deployment of a High-Availability (HA) LAMP stack utilizing a 4-node cluster deployed on OpenStack (MicroStack). The project required establishing internal Virtual Private Cloud (VPC) networks, deploying isolated security groups, navigating strict hypervisor resource schedulers, diagnosing multi-layer network encapsulation blockages (MTU/VXLAN), and configuring an Nginx reverse-proxy load balancer to distribute traffic across a dynamic PHP/MySQL backend.

Cluster Architecture

Node Name	Role	Private IP	Public (Floating) IP
ha-db-node	Database Backend (MySQL)	192.168.100.59	-
ha-web-node-1	Apache Web Server	192.168.100.249	-
ha-web-node-2	Apache Web Server	192.168.100.12	-
ha-lb-node	Nginx Load Balancer / Bastion	192.168.100.67	10.20.20.133

2. Security Architecture (Security Groups)

To enforce strict zero-trust network isolation, dedicated Security Groups were created for each tier of the architecture. Instead of relying on the default OpenStack group, specific rules were engineered to allow only necessary ingress/egress traffic:

```
microstack.openstack security group create ha-lb-sg
microstack.openstack security group create ha-web-sg
microstack.openstack security group create ha-db-sg
```

With the groups created, specific port mapping rules were applied. The load balancer was opened to the public on ports 80, 443, and 22. To secure the backend, the web nodes were configured to only accept HTTP traffic coming directly from the Load Balancer's security group, and the database was locked down to only accept MySQL connections (Port 3306) coming from the Web Nodes' security group:

```

microstack.openstack security group rule create --protocol tcp --dst-port
80:80 ha-lb-sg
microstack.openstack security group rule create --protocol tcp --dst-port
443:443 ha-lb-sg
microstack.openstack security group rule create --protocol tcp --dst-port
22:22 ha-lb-sg

microstack.openstack security group rule create --protocol tcp --dst-port
80:80 --remote-group ha-lb-sg ha-web-sg
microstack.openstack security group rule create --protocol tcp --dst-port
22:22 ha-web-sg

microstack.openstack security group rule create --protocol tcp --dst-port
3306:3306 --remote-group ha-web-sg ha-db-sg
microstack.openstack security group rule create --protocol tcp --dst-port
22:22 ha-db-sg

```

```
joe@openstack1:~$ microstack.openstack security group list
```

ID	Name	Description	Project	Tags
3431a783-c48e-4955-95dc-018f5db6ba4a	ha-web-sg	ha-web-sg	74ee6b1f2f1546cab824670048dc7169	[]
393b9a09-d196-4bf9-8dbc-78f529cc8856	lab-sg	Lab-4-security-group	74ee6b1f2f1546cab824670048dc7169	[]
46071c8b-9a3c-4a1b-98b4-847c9b8c83e6	default	Default security group	74ee6b1f2f1546cab824670048dc7169	[]
550dfffc1-0239-4dad-9925-2ca59bf3f23c	web-sg	Group-for-Web-Server-VMs	74ee6b1f2f1546cab824670048dc7169	[]
79560ec0-b3ed-4e9a-ae4c-170927449301	default	Default security group	f9b1d42023dc49cb98d515b4482f6d56	[]
7e568e57-c4b7-4eb8-b4ab-69fc0ab45448	default	Default security group	50e3d1e8818b42d7b786f85955b83c38	[]
8835a3d9-3a82-4918-a6a3-4e294d49d5b8	private-sg	Group-for-Web-Server-VMs	74ee6b1f2f1546cab824670048dc7169	[]
dc9fbb72-511e-4cf7-b739-25ebf77b3c79	ha-lb-sg	ha-lb-sg	74ee6b1f2f1546cab824670048dc7169	[]
e2e6a44e-c11a-4ef2-994d-aa745acb7955	ha-db-sg	ha-db-sg	74ee6b1f2f1546cab824670048dc7169	[]

```
joe@openstack1:~$
```

Figure 1: Security Group overview showing the isolated groups created for the web and database tiers.

```
joe@openstack1:~$ microstack.openstack security group rule list ha-lb-sg
```

ID	IP Protocol	Ethertype	IP Range	Port Range	Remote Security Group
05d24fb1-62bb-484e-ab53-663d573aa7fe	None	IPv4	0.0.0.0/0		None
1b957c23-82bb-487c-9f9e-00e026bcbf76	tcp	IPv4	0.0.0.0/0	80:80	None
1be5ef49-8eb1-4d5f-b11e-3e7448788d86	tcp	IPv4	0.0.0.0/0	22:22	None
40724ed3-5101-4d74-85dc-a2370f7d1396	tcp	IPv4	0.0.0.0/0	443:443	None
445bd1f3-b004-47cb-af31-970ce6e91a04	None	IPv6	::/0		None

```
joe@openstack1:~$ microstack.openstack security group rule list ha-web-sg
```

ID	IP Protocol	Ethertype	IP Range	Port Range	Remote Security Group
19b99ffc-ee81-447e-818b-c32e52ba3c17	tcp	IPv4	0.0.0.0/0	80:80	dc9fbb72-511e-4cf7-b739-25ebf77b3c79
2062535b-970a-422f-a737-f54181b9eff2	None	IPv4	0.0.0.0/0		None
53e0651d-cfcd-4645-9258-0997110ba2af	tcp	IPv4	0.0.0.0/0	22:22	None
e6ec585d-2b8b-4864-9cbb-ded8d4d177ed	None	IPv6	::/0		None

```
joe@openstack1:~$ microstack.openstack security group rule list ha-db-sg
```

ID	IP Protocol	Ethertype	IP Range	Port Range	Remote Security Group
0b99a51f-5cac-43dd-b43f-c69ddb59539	None	IPv6	::/0		None
24c1ef0b-e382-48c5-bea3-5eb107b8cca6	tcp	IPv4	0.0.0.0/0	22:22	None
996b1fe3-1696-4059-83a2-1f67136ccef6	None	IPv4	0.0.0.0/0		None
c7838f7f-51a3-4521-b324-86a5522fa5a5	tcp	IPv4	0.0.0.0/0	3306:3306	3431a783-c48e-4955-95dc-018f5db6ba4a

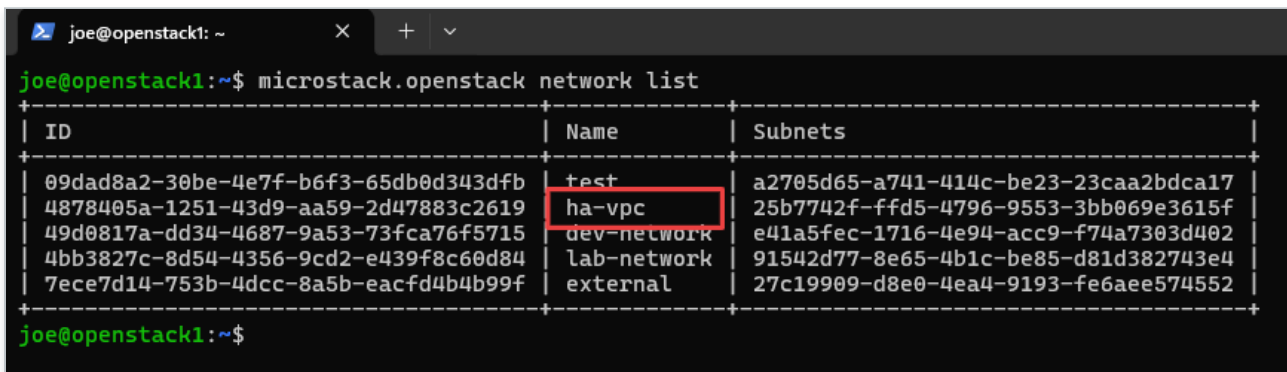
```
joe@openstack1:~$
```

Figure 2: Detailed protocol and port mapping configurations enforcing secure tier-to-tier access.

3. Network Isolation (VPC & Subnets)

The backend fleet was deployed into a private network (VPC). A Virtual Private Cloud is considered "isolated" because it operates as a logically separated network boundary with its own private IP address space (192.168.100.x). By default, instances inside this subnet lack a direct internet gateway—meaning they cannot be routed to from the outside world, effectively shielding them from public exposure.

```
microstack.openstack network create ha-vpc
microstack.openstack subnet create --network ha-vpc --subnet-range
192.168.100.0/24 ha-subnet
```

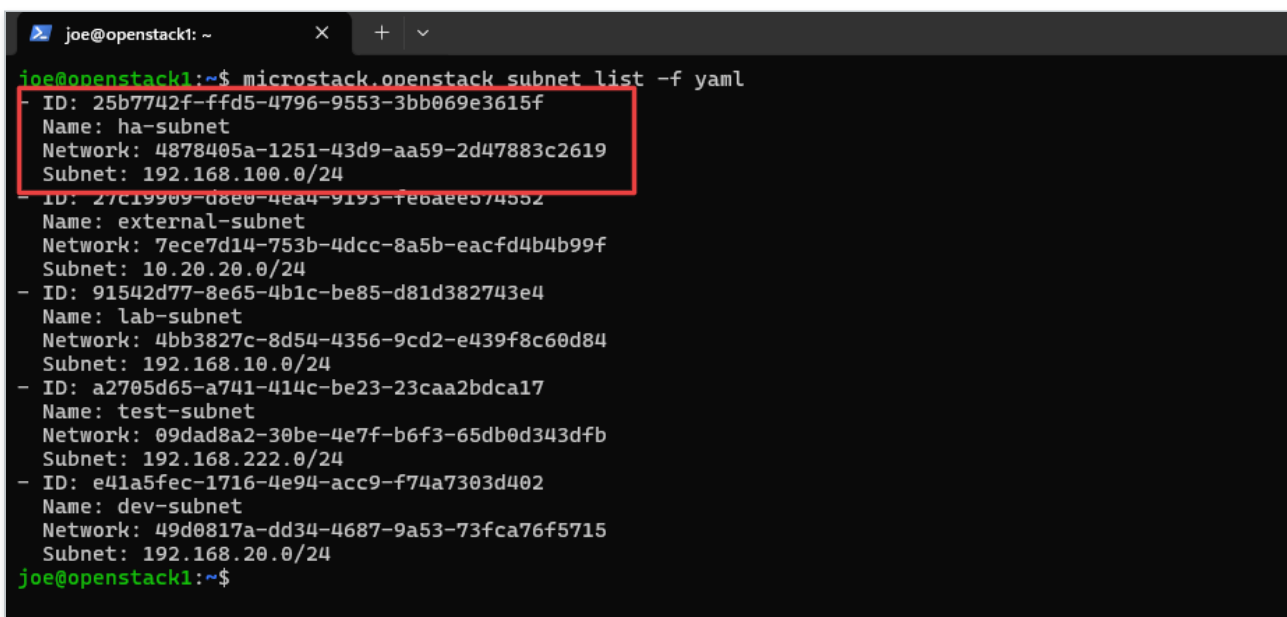


```
joe@openstack1: ~
joe@openstack1:~$ microstack.openstack network list
```

ID	Name	Subnets
09dad8a2-30be-4e7f-b6f3-65db0d343dfb	test	a2705d65-a741-414c-be23-23caa2bdca17
4878405a-1251-43d9-aa59-2d47883c2619	ha-vpc	25b7742f-ffd5-4796-9553-3bb069e3615f
49d0817a-dd34-4687-9a53-73fca76f5715	dev-network	e41a5fec-1716-4e94-acc9-f74a7303d402
4bb3827c-8d54-4356-9cd2-e439f8c60d84	lab-network	91542d77-8e65-4b1c-be85-d81d382743e4
7ece7d14-753b-4dcc-8a5b-eacfd4b4b99f	external	27c19909-d8e0-4ea4-9193-fe6aee574552

```
joe@openstack1:~$
```

Figure 3: Verification of the isolated ha-vpc network creation.



```
joe@openstack1: ~
joe@openstack1:~$ microstack.openstack subnet list -f yaml
```

```
- ID: 25b7742f-ffd5-4796-9553-3bb069e3615f
  Name: ha-subnet
  Network: 4878405a-1251-43d9-aa59-2d47883c2619
  Subnet: 192.168.100.0/24
- ID: 27c19909-d8e0-4ea4-9193-fe6aee574552
  Name: external-subnet
  Network: 7ece7d14-753b-4dcc-8a5b-eacfd4b4b99f
  Subnet: 10.20.20.0/24
- ID: 91542d77-8e65-4b1c-be85-d81d382743e4
  Name: lab-subnet
  Network: 4bb3827c-8d54-4356-9cd2-e439f8c60d84
  Subnet: 192.168.10.0/24
- ID: a2705d65-a741-414c-be23-23caa2bdca17
  Name: test-subnet
  Network: 09dad8a2-30be-4e7f-b6f3-65db0d343dfb
  Subnet: 192.168.222.0/24
- ID: e41a5fec-1716-4e94-acc9-f74a7303d402
  Name: dev-subnet
  Network: 49d0817a-dd34-4687-9a53-73fca76f5715
  Subnet: 192.168.20.0/24
joe@openstack1:~$
```

Figure 4: Verification of the 192.168.100.x private subnet configuration.

4. Instance Provisioning & Overcommitment Resolution

VM Creation & Scheduler Limits

The cluster nodes were instantiated using the OpenStack CLI. To bypass an OpenStack metadata timeout error that resulted in empty `authorized_keys` files, the instances were deployed using the `--config-drive true` flag, physically mounting the SSH keys during the boot sequence:

```
microstack.openstack server create --flavor m1.small --image ubuntu-24.04 -
-network ha-vpc --key-name ha-lamp-key --security-group ha-lb-sg --config-
drive true ha-lb-node
microstack.openstack server create --flavor m1.small --image ubuntu-24.04 -
-network ha-vpc --key-name ha-lamp-key --security-group ha-web-sg --config-
drive true ha-web-node-1
microstack.openstack server create --flavor m1.small --image ubuntu-24.04 -
-network ha-vpc --key-name ha-lamp-key --security-group ha-web-sg --config-
drive true ha-web-node-2
microstack.openstack server create --flavor m1.small --image ubuntu-24.04 -
-network ha-vpc --key-name ha-lamp-key --security-group ha-db-sg --config-
drive true ha-db-node
```

During instantiation, the OpenStack Nova Scheduler returned an HTTP 500: `NoValidHost` exception. Investigation revealed that the scheduler enforces a strict 1:1 disk allocation ratio. Launching the fourth node pushed the theoretical reserved disk space above the hypervisor's capacity, even though physical space was available. This was resolved by injecting an override into `nova.conf.d/override.conf`:

```
[DEFAULT]
disk_allocation_ratio = 100.0
ram_allocation_ratio = 100.0
cpu_allocation_ratio = 100.0
```

```
joe@openstack1: ~
joe@openstack1:~$ microstack.openstack server list
```

ID	Name	Status	Networks	Image	Flavor
d89b462b-d64c-4577-893d-a012c3266d09	ha-lb-node	ACTIVE	ha-vpc=192.168.100.67	ubuntu-24.04	m1.small
e9e4a9a8-e009-43ee-98cf-6b3e785b8ae0	ha-web-node-2	ACTIVE	ha-vpc=192.168.100.30	ubuntu-24.04	m1.small
8aa14aa1-cb9b-4c04-b3e4-4434f0378fc1	ha-web-node-1	ACTIVE	ha-vpc=192.168.100.34	ubuntu-24.04	m1.small
0a53a7f9-a64b-4bcd-8227-54483ccdab86	ha-db-node	ACTIVE	ha-vpc=192.168.100.179	ubuntu-24.04	m1.small

```
joe@openstack1:~$
```

Figure 5: Active Server List showing all 4 nodes actively running after bypassing the scheduler limit.

Floating IP Assignment & Edge Routing

To expose the load balancer to the external network while keeping the web and database nodes completely private, a Floating IP was generated and mapped exclusively to ha-lb-node:

```
microstack.openstack floating ip create external
microstack.openstack server add floating ip ha-lb-node 10.20.20.133
```

```
joe@openstack1: ~
joe@openstack1:~$ microstack.openstack server add floating ip ha-lb-node 10.20.20.133
joe@openstack1:~$ microstack.openstack server list
```

ID	Name	Status	Networks	Image	Flavor
d89b462b-d64c-4577-893d-a012c3266d09	ha-lb-node	ACTIVE	ha-vpc=192.168.100.67, 10.20.20.133	ubuntu-24.04	m1.small
e9e4a9a8-e009-43ee-98cf-6b3e785b8ae0	ha-web-node-2	ACTIVE	ha-vpc=192.168.100.30	ubuntu-24.04	m1.small
8aa14aa1-cb9b-4c04-b3e4-4434f0378fc1	ha-web-node-1	ACTIVE	ha-vpc=192.168.100.34	ubuntu-24.04	m1.small
0a53a7f9-a64b-4bcd-8227-54483ccdab86	ha-db-node	ACTIVE	ha-vpc=192.168.100.179	ubuntu-24.04	m1.small

```
joe@openstack1:~$
```

Figure 6: Server list displaying ha-lb-node successfully bound to the external 10.20.20.133 floating IP.

5. Initial Access & The "Nested Cloud" Network Trap

With the Floating IP attached to the Load Balancer, an initial SSH connection was successfully established from the host machine into the private cloud.

```
joe@openstack1:~$ ssh -i ha-lamp-key.pem -o PubkeyAcceptedKeyTypes=+ssh-rsa -o HostKeyAlgorithms=+ssh-rsa ubuntu@10.20.20.133
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-106-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 1 19:02:45 UTC 2026

System load: 0.0          Processes:      100
Usage of /:  10.4% of 18.33GB Users logged in: 0
Memory usage: 10%        IPv4 address for ens3: 192.168.100.58
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

77 updates can be applied immediately.
57 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Fri May 1 15:21:39 2026 from 10.20.20.1
ubuntu@ha-lb-node:~$
```

Figure 7: Terminal window confirming successful SSH access into the Load Balancer via the floating IP.

However, once inside the VM, outbound network requests (such as `apt update`) were timing out. This represented a complex networking failure typical of nested virtualization (running a cloud inside a VM). Resolving this required configuring all three layers of the network architecture simultaneously:

1. Virtual Firewall (OpenStack): The Security Groups were updated to explicitly allow outbound TCP traffic and inbound ICMP pings.

2. VM Network Settings (Guest OS): Because OpenStack uses VXLAN tunnels to route private VPC traffic, it adds overhead to the data packets. The default MTU of 1500 was too large and caused packet fragmentation/dropping. The packet size was forcefully reduced to 1300, and reliable external DNS (8.8.8.8) was injected into every instance:

```
sudo ip link set dev ens3 mtu 1300
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

3. Host Gateway Bridging (Hypervisor): Due to the OpenStack hypervisor residing on a Windows machine, the host OS had no native route to the virtual 10.20.20.x subnet. To allow internet traffic to flow properly to the Load Balancer, a static route and IP masquerading were added to the physical host routing table:

```
route add 10.20.20.0 mask 255.255.255.0 192.168.230.129
```

6. Web Node Configuration & Reverse Proxy

Apache Installation

With the network constraints resolved, Apache2 was successfully installed across the backend fleet:

```
sudo apt update -o Acquire::ForceIPv4=true
sudo apt install apache2 -y -o Acquire::ForceIPv4=true
```

Nginx Load Balancer Setup

On the ha-lb-node, the default Nginx site was removed and replaced with a custom upstream routing block pointing to the private VPC IPs of the newly provisioned Apache web nodes.

What is a Reverse Proxy?

Unlike a standard forward proxy (which protects outgoing traffic from clients), a reverse proxy sits securely in front of backend servers. When a user requests a webpage, the request goes to the Nginx proxy first. Nginx then fetches the page from the hidden Apache web nodes and returns it to the user. This masks the backend servers' true identities, prevents direct external access, and allows Nginx to distribute incoming traffic seamlessly.

```
upstream backend_fleet {
    server 192.168.100.249;
    server 192.168.100.12;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend_fleet;
    }
}
```

Load Balancing Algorithm: By default, Nginx utilizes a *Round-Robin* algorithm. This means the load balancer acts as a strict traffic cop, routing the first incoming user to Node 1, the second user

to Node 2, the third back to Node 1, and so on. This ensures an even distribution of CPU and memory load across the backend fleet.

```
ubuntu@ha-lb-node: ~$ curl localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;

        background-color: #D8DBE2;

        font-family: Ubuntu, Verdana, sans-serif;
        font-size: 11pt;
        text-align: center;
      }

      div.main_page {
        position: relative;
        display: table;

        width: 800px;

        margin-bottom: 3px;
        margin-left: auto;
```

Figure 8: Successful HTTP response from the load balancer fetching the backend Apache nodes.

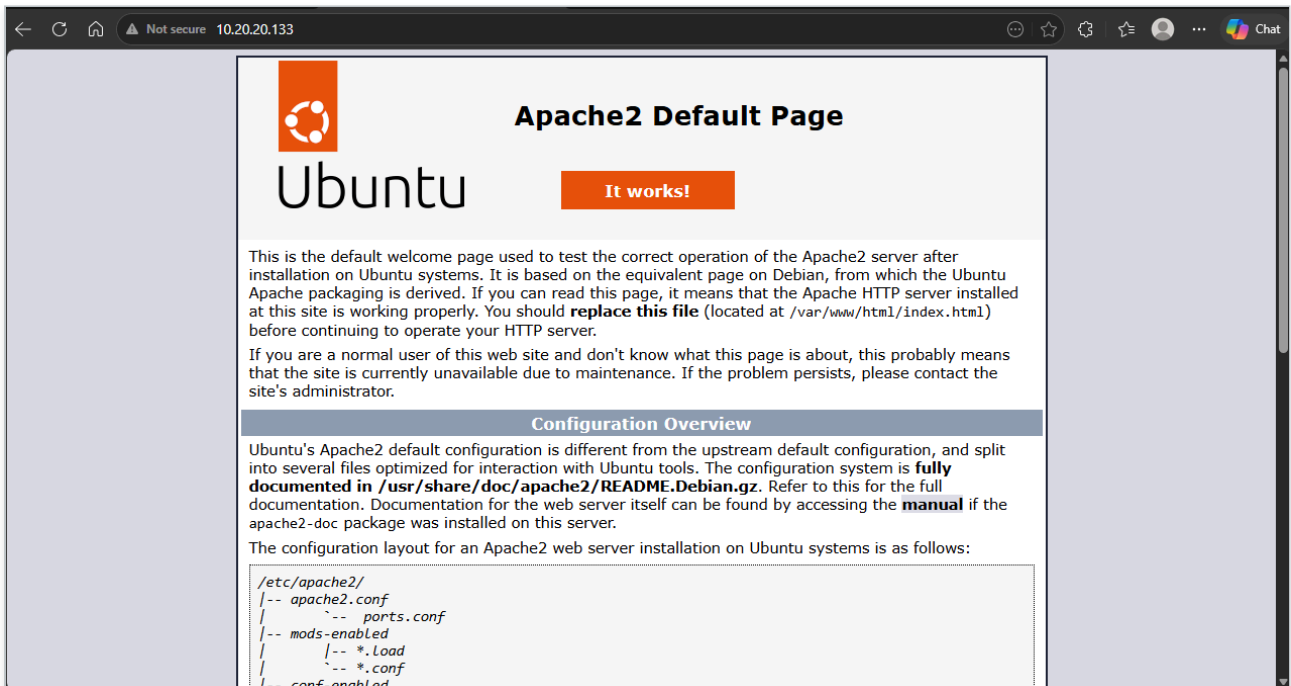


Figure 9: Verification of the Nginx Reverse Proxy accessed from the external Windows host browser.

7. Database Backend Configuration (MySQL)

Bastion Host Pivoting

Because the database node (192.168.100.59) resides exclusively on the private subnet with no public IP, direct SSH access was impossible. To securely configure the database, the Nginx Load Balancer was utilized as a **Bastion Host (Jump Server)**. A direct tunnel was established using an SSH ProxyCommand, elegantly bypassing the external routing limitations:

```
ssh -o ProxyCommand="ssh -W %h:%p -i ha-lamp-key.pem -o
PubkeyAcceptedKeyTypes=+ssh-rsa -o HostKeyAlgorithms=+ssh-rsa
ubuntu@10.20.20.133" -i ha-lamp-key.pem -o PubkeyAcceptedKeyTypes=+ssh-rsa
-o HostKeyAlgorithms=+ssh-rsa ubuntu@192.168.100.59
```

Database Network Exposure

After applying the standard VPC MTU and DNS configurations (as established in Section 5), MySQL Server was installed. By default, MySQL binds strictly to the local loopback interface (127.0.0.1). To allow the Apache web nodes to communicate with the database, the configuration file was modified to bind to 0.0.0.0 over Port 3306.

```
sudo systemctl restart mysql
```

```
ubuntu@ha-db-node: ~$ cat /etc/mysql/mysql.conf.d/mysqld.cnf
#
# The MySQL database server configuration file.
#
# One can use all long options that the program supports.
# Run program with --help to get a list of available options and with
# --print-defaults to see which it would actually understand and use.
#
# For explanations see
# http://dev.mysql.com/doc/mysql/en/server-system-variables.html
#
# Here is entries for some specific programs
# The following values assume you have at least 32M ram

[mysqld]
#
# * Basic Settings
#
user                = mysql
pid-file            = /var/run/mysqld/mysqld.pid
socket              = /var/run/mysqld/mysqld.sock
port                = 3306
datadir             = /var/lib/mysql

# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_tmpdir
tmpdir              = /tmp
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address        = 0.0.0.0
mysqlx-bind-address = 127.0.0.1
#
```

Figure 10: Successful modification of the MySQL bind address and service restart via the Bastion tunnel.

8. Application Layer Integration (PHP)

Troubleshooting: Apache Module Conflicts

After installing PHP, the web servers initially returned raw PHP source code instead of executing it. This occurred because Ubuntu 24.04 Apache utilizes the `mpm_event` module, which conflicts with standard PHP execution. The event module was explicitly disabled, the `mpm_prefork` engine was ignited, and the PHP 8.3 module was enabled:

```
sudo a2dismod mpm_event
sudo a2enmod mpm_prefork
sudo a2enmod php8.3
sudo systemctl restart apache2
```

Remote Database Authentication & Bash Traps

To establish a secure database connection, a remote web user had to be created. An attempt to create this user via a single-line bash command failed with a `-bash: !': event not found` error. This was caused by the bash shell attempting to parse the exclamation mark in the database password (`LampPass123!`) as a history expansion variable.

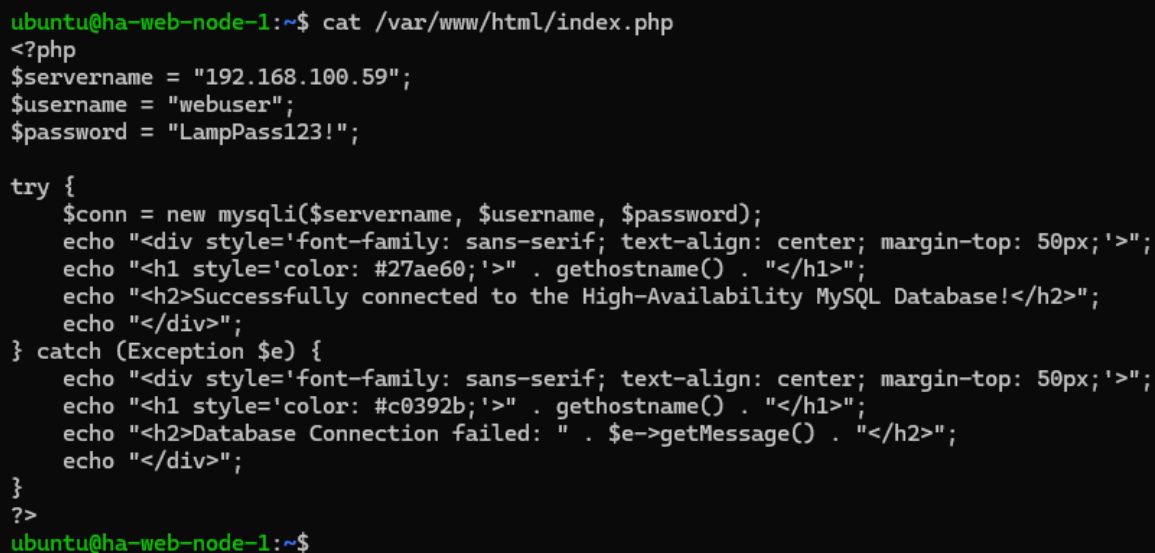
Solution: The bash terminal was bypassed by dropping directly into the MySQL interactive shell on the database node, securely executing the remote user provisioning:

```
sudo mysql

CREATE USER 'webuser'@'%' IDENTIFIED BY 'LampPass123!';
GRANT ALL PRIVILEGES ON *.* TO 'webuser'@'%';
FLUSH PRIVILEGES;
exit
```

Final Deployment & High-Availability Validation

A dynamic PHP validation script was deployed to both web nodes to dynamically grab the internal hostname and connect to the Database Node (192.168.100.59) using the webuser credentials.

A terminal window on a node named 'ubuntu@ha-web-node-1' shows the execution of a PHP script. The script defines database connection parameters (\$servername, \$username, \$password) and uses a try-catch block to attempt a MySQL connection. If successful, it outputs a green message; if it fails, it outputs a red message. The prompt returns to the shell after execution.

```
ubuntu@ha-web-node-1:~$ cat /var/www/html/index.php
<?php
$servername = "192.168.100.59";
$username = "webuser";
$password = "LampPass123!";

try {
    $conn = new mysqli($servername, $username, $password);
    echo "<div style='font-family: sans-serif; text-align: center; margin-top: 50px;'>";
    echo "<h1 style='color: #27ae60;'>" . gethostname() . "</h1>";
    echo "<h2>Successfully connected to the High-Availability MySQL Database!</h2>";
    echo "</div>";
} catch (Exception $e) {
    echo "<div style='font-family: sans-serif; text-align: center; margin-top: 50px;'>";
    echo "<h1 style='color: #c0392b;'>" . gethostname() . "</h1>";
    echo "<h2>Database Connection failed: " . $e->getMessage() . "</h2>";
    echo "</div>";
}
?>
ubuntu@ha-web-node-1:~$
```

Figure 11: Deploying the dynamic MySQL connection test script to the Apache web nodes.

Upon accessing the Nginx Load Balancer's public IP via a private browser window, the PHP script successfully executed. Subsequent refreshes confirmed that the Nginx reverse proxy was actively distributing the HTTP load (Round-Robin) across ha-web-node-1 and ha-web-node-2, while both successfully maintained connections to the backend database.

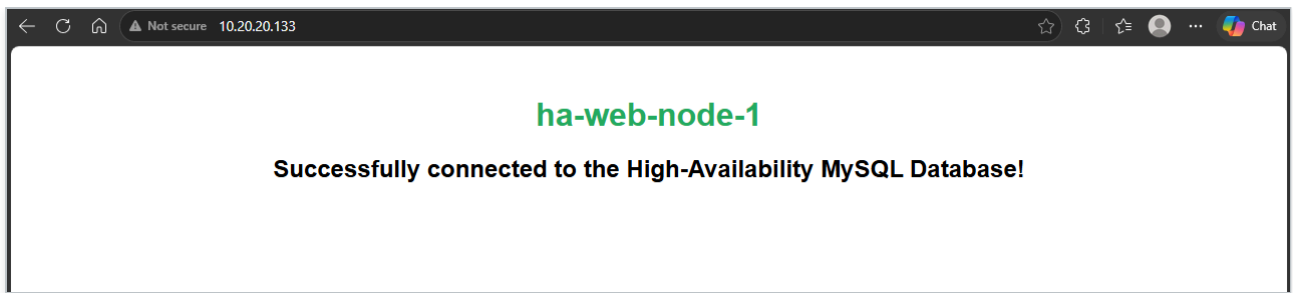


Figure 12: Successful dynamic database connection proving complete end-to-end load-balanced LAMP architecture.

9. Conclusion & Security Architecture Analysis

Project Requirements Fulfillment

This deployment successfully fulfills all requirements for the Advanced Cloud Deployment mini-project:

- **Deployed a LAMP Environment Across VMs:** The architecture successfully split the LAMP stack across 4 distinct virtual machines, utilizing Nginx as the frontend proxy, Apache/PHP as the application layer, and MySQL as the data layer.
- **Configured Web and Database Connectivity:** Dynamic PHP scripts were engineered to successfully traverse the internal subnet, authenticate with a dedicated remote database user, and return active connection statuses to the client.
- **Explored Service Dependencies:** The project required navigating complex dependencies, including rectifying Apache MPM module conflicts to execute PHP, and modifying MySQL bind-addresses to allow network-level SQL queries.
- **Analyzed Performance in a Virtualized Environment:** Deep performance and networking analysis was conducted to discover and resolve MTU packet dropping caused by OpenStack's nested VXLAN overhead, alongside bypassing hypervisor resource overcommitment locks.

VPC Security & Zero-Trust Isolation

The architecture was engineered with a strict security-first posture. By utilizing OpenStack VPCs and isolated Security Groups, the application backend (Web Nodes and Database Node) operates completely in the dark. **They possess no public IP addresses.** The only entity capable of receiving external traffic is the Nginx Load Balancer. Consequently, external threat actors cannot directly access, ping, or SSH into the application or data layers. Furthermore, the internal Security Group rules strictly regulate traffic flow (e.g., the database will only accept traffic originating from the web servers' security group). All administrative access requires utilizing the Load Balancer as a secure Bastion Jump Host, fulfilling the highest standards of cloud network security.